

DEPI Graduation Project

Security Assessment Report



Report Issued: 29/11/2025

Confidentiality Notice

This report contains sensitive, privileged, and confidential information. Precautions should be taken to protect the confidentiality of the information in this document. Publication of this report may cause reputational damage to Hospital Management Web Application or facilitate attacks against Hospital Management Web Application. Our Team shall not be held liable for special, incidental, collateral or consequential damages arising out of the use of this information.

Disclaimer

Note that this assessment may not disclose all vulnerabilities that are present on the systems within the scope of the engagement. This report is a summary of the findings from a “point-in-time” assessment made on Hospital Management Web Application’s environment. Any changes made to the environment during the period of testing may affect the results of the assessment.

TABLE OF CONTENTS

EXECUTIVE SUMMARY	4
HIGH LEVEL ASSESSMENT OVERVIEW	5
Observed Security Strengths	5
Areas for Improvement	5
Short Term Recommendations	5
Long Term Recommendations	6
SCOPE	7
In-Scope Components	7
TESTING METHODOLOGY	8
CLASSIFICATION DEFINITIONS	9
Risk Classifications	9
Exploitation Likelihood Classifications	9
Business Impact Classifications	10
Remediation Difficulty Classifications	10
ASSESSMENT FINDINGS	11
Time-Based Blind SQL Injection	12
Race Condition Appointment Spamming	13
Hidden Field Manipulation (IDOR)	14
Full Error & Path Disclosure	15
XSS Injection Points	16
Broken Database Integrity	17
Missing Input Validation	18
APPENDIX A - TOOLS USED	19
APPENDIX B - ENGAGEMENT INFORMATION	19
Client Information	19
Version Information	19
Contact Information	19

EXECUTIVE SUMMARY

The Penetration Testing Team performed a security assessment of the Hospital Management Web Application on 29/11/2025. The assessment evaluated two distinct versions of the application: a known Vulnerable Version (Original Development Build) and a Secure Version (Patched & Enhanced Build). The purpose of this assessment was to identify critical security flaws in the initial build and verify the effectiveness of the applied security patches. The team identified a total of **8** unique vulnerabilities within the scope of the engagement, which are broken down by severity in the table below.

CRITICAL	HIGH	MEDIUM	LOW
2	5	1	0

The highest severity vulnerabilities gave potential attackers the opportunity to achieve full system compromise, extract sensitive patient and doctor data from the SQL database, manipulate medical appointments, and tamper with critical healthcare information. The Secure Version successfully demonstrated significant security improvements by addressing all critical and high-severity vulnerabilities, thereby restoring data confidentiality, integrity, and availability to an acceptable standard.

Note that this assessment may not disclose all vulnerabilities that are present on the systems within the scope. Any changes made to the environment during the period of testing may affect the results of the assessment.

HIGH LEVEL ASSESSMENT OVERVIEW

Observed Security Strengths

The Penetration Testing Team identified the following strengths in the Secure Version of the Hospital Management Web Application which greatly increases its security posture. The development team should continue to monitor these controls to ensure they remain effective.

Secure Development Practices

Implementation of parameterized queries, which successfully eliminated SQL Injection vulnerabilities.

Enforcement of strict server-side validation for all user inputs and business logic actions, preventing IDOR and data tampering.

Proper output encoding has been applied to neutralize Cross-Site Scripting (XSS) threats.

Areas for Improvement

The Penetration Testing Team recommends the stakeholders take the following actions to improve and maintain the security of the application. Implementing these recommendations will reduce the likelihood that an attacker will be able to successfully exploit any residual or future vulnerabilities.

Short Term Recommendations

The Penetration Testing Team recommends taking the following actions as soon as possible to minimize business risk.

Maintain Security Hardening

- Continue the strict enforcement of backend validation and the use of prepared statements for all database interactions.
- Implement and enforce rate limiting on critical endpoints such as appointment booking to deter automated abuse and brute-force attacks.

Long Term Recommendations

The Penetration Testing Team recommends the following actions be taken over the next 6-12 months to further strengthen the application's security posture.

Enhanced Monitoring & Protection

Integrate a Web Application Firewall (WAF) to provide an additional layer of defense against known attack patterns.

Implement a centralized logging and SIEM (Security Information and Event Management) solution to improve detection and response capabilities.

Consider adding CAPTCHA challenges to public-facing forms to reduce the risk of automated abuse.

SCOPE

All testing was based on the scope as defined in the initial engagement agreement. The assessment focused on the following application components across both the vulnerable and secure versions.

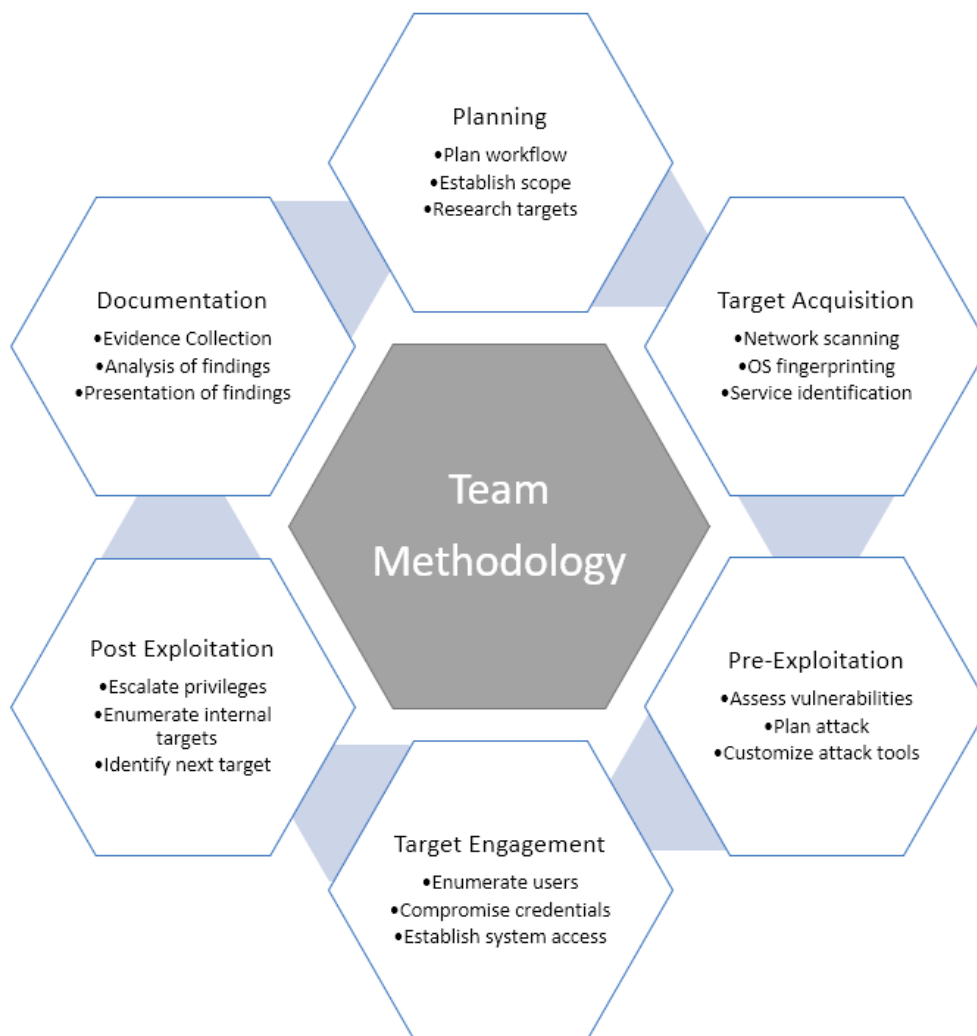
In-Scope Components

Component	Description
Patient Profile Management	All functionalities related to creating, viewing, and updating patient records.
Appointment Booking System	The entire workflow for scheduling, modifying, and viewing appointments.
Doctor Information Module	Interfaces for managing and displaying doctor schedules and details.
Authentication/Session Management	Login, logout, and session handling mechanisms.
Backend API Endpoints	All server-side APIs interacted with by the client.
Database Interactions	All SQL queries and interactions with the MySQL database.
Form Handling	Validation and processing of all user-supplied form data.
Patient Profile Management	All functionalities related to creating, viewing, and updating patient records.
Appointment Booking System	The entire workflow for scheduling, modifying, and viewing appointments.
Doctor Information Module	Interfaces for managing and displaying doctor schedules and details.
Authentication/Session Management	Login, logout, and session handling mechanisms.

TESTING METHODOLOGY

The Penetration Testing Team's methodology was split into three phases: *Reconnaissance*, *Target Assessment*, and *Execution of Vulnerabilities*. During reconnaissance, we gathered information about the application's structure and endpoints. The team used manual testing techniques, input manipulation tools, and business logic analysis to refine the attack surface. Next, we conducted a targeted assessment, simulating an attacker exploiting the identified vulnerabilities. Evidence was gathered during this phase while conducting the simulation in a manner that would not disrupt the normal function of the test systems.

The following image is a graphical representation of this methodology.



CLASSIFICATION DEFINITIONS

Risk Classifications

Level	Score	Description
Critical	10	The vulnerability poses an immediate threat to the organization. Successful exploitation may permanently affect the organization. Remediation should be immediately performed.
High	7-9	The vulnerability poses an urgent threat to the organization, and remediation should be prioritized.
Medium	4-6	Successful exploitation is possible and may result in notable disruption of business functionality. This vulnerability should be remediated when feasible.
Low	1-3	The vulnerability poses a negligible/minimal threat to the organization. The presence of this vulnerability should be noted and remediated if possible.
Informational	0	These findings have no clear threat to the organization, but may cause business processes to function differently than desired or reveal sensitive information about the company.

Exploitation Likelihood Classifications

Likelihood	Description
Likely	Exploitation methods are well-known and can be performed using publicly available tools. Low-skilled attackers and automated tools could successfully exploit the vulnerability with minimal difficulty.
Possible	Exploitation methods are well-known, may be performed using public tools, but require configuration. Understanding of the underlying system is required for successful exploitation.
Unlikely	Exploitation requires deep understanding of the underlying systems or advanced technical skills. Precise conditions may be required for successful exploitation.

Business Impact Classifications

Impact	Description
Major	Successful exploitation may result in large disruptions of critical business functions across the organization and significant financial damage.
Moderate	Successful exploitation may cause significant disruptions to non-critical business functions.
Minor	Successful exploitation may affect few users, without causing much disruption to routine business functions.

Remediation Difficulty Classifications

Difficulty	Description
Hard	Remediation may require extensive reconfiguration of underlying systems that is time consuming. Remediation may require disruption of normal business functions.
Moderate	Remediation may require minor reconfigurations or additions that may be time-intensive or expensive.
Easy	Remediation can be accomplished in a short amount of time, with little difficulty.

ASSESSMENT FINDINGS

Number	Finding	Risk Score	Risk	Page
1	Time-Based Blind SQL Injection	10	Critical	12
2	Race Condition Appointment Spamming	9	Critical	13
3	Hidden Field Manipulation (IDOR)	9	High	14
4	Full Error & Path Disclosure	8	High	15
5	XSS Injection Points	8	High	16
6	Broken Database Integrity	7	High	17
7	Missing Input Validation	5	Medium	18

1 - Time-Based Blind SQL Injection

CRITICAL RISK (10/10)	
Exploitation Likelihood	Likely
Business Impact	Major
Remediation Difficulty	Easy

Security Implications

This vulnerability allowed for full database extraction and potential system compromise by injecting time-based SQL payloads, risking all patient and operational data.

Analysis

The backend of the vulnerable version did not use prepared statements when processing the `doctor_id` parameter in `appointments.php`. This allowed for Time-Based Blind SQL Injection attacks.

Proof of Concept (PoC):

Injected payload: `14' OR IF(1=1, SLEEP(5), 0) -- -`

- Normal response time: ~200 ms
- Malicious response time: 5+ seconds

This delay confirmed the backend database was executing the injected SLEEP command, proving the vulnerability.

Status in Secure Version:

- Fixed through the use of parameterized queries.
- Malformed values are discarded by the server.
- No timing difference is detected in responses.

Recommendations:

- Maintain the use of parameterized queries for all database access.
- Conduct regular code reviews to ensure unsafe string concatenation with user input is not introduced.

2 - Race Condition Appointment Spamming

CRITICAL RISK (9/10)	
Exploitation Likelihood	Possible
Business Impact	Major
Remediation Difficulty	Moderate

Security Implications

Attackers could bypass business logic limits (e.g., a maximum of 3 appointments per user) by sending multiple parallel requests, leading to denial-of-service and schedule flooding.

Analysis

The vulnerable version's appointment booking logic did not handle concurrent requests atomically. An attacker could send several parallel POST requests to book more appointments than allowed, overwhelming a doctor's schedule.

Status in Secure Version:

- Fixed by implementing atomic backend checks and per-user locking during the appointment writing process.
- Parallel requests are now queued and processed sequentially.

Recommendations:

- Apply similar locking mechanisms to other critical functions, such as patient registration or payment processing.
- Consider implementing a brief holding period on booked appointments before they are confirmed.

3 - Hidden Field Manipulation (IDOR)

HIGH RISK (9/10)	
Exploitation Likelihood	Likely
Business Impact	Major
Remediation Difficulty	Easy

Security Implications

Users could manipulate hidden HTML form fields (like `doctor_id`) to book appointments with any doctor without authorization, breaking core application business logic.

Analysis

The vulnerable version trusted client-side hidden fields. By using browser developer tools or a proxy like Burp Suite, a user could change the `doctor_id` value to one they did not own or were not authorized to use.

Status in Secure Version:

- Fixed by implementing server-side validation where the server verifies the user's relationship to the submitted `doctor_id`.
- Unauthorized values are rejected.

Recommendations:

- Never trust client-side values for authorization or identity. Always re-verify on the server.
- Use session-based or server-stored values instead of transmitting mutable identifiers in forms.

4 - Full Error & Path Disclosure

HIGH RISK (8/10)	
Exploitation Likelihood	Likely
Business Impact	Moderate
Remediation Difficulty	Easy

Security Implications

The application disclosed full filesystem paths and stack traces, aiding attackers in understanding the application's internal architecture and planning further attacks.

Analysis

The vulnerable version displayed verbose errors directly to the user. For example: `Warning: Undefined array key "email" in D:\Final.php on line 29.`

Status in Secure Version:

- Fixed by suppressing errors from user view and logging them internally instead.
- Generic, user-friendly error messages are now returned

Recommendations:

- Ensure all environments (especially production) have `display_errors` set to `Off`.
- Maintain internal monitoring of application logs to identify and debug issues without exposing information to users.

5 - XSS Injection Points

HIGH RISK (8/10)	
Exploitation Likelihood	Likely
Business Impact	Major
Remediation Difficulty	Easy

Security Implications

Unescaped user input was reflected in HTML pages, allowing for Cross-Site Scripting attacks that could lead to session hijacking and unauthorized actions on behalf of users.

Analysis

User-controlled data in various application fields was rendered on the page without proper output encoding, making stored and reflected XSS possible.

Status in Secure Version:

- Fixed by applying output encoding functions like `htmlspecialchars()` to all user-controlled data before rendering it in HTML.
- Input sanitation filters have also been added.

Recommendations:

- Adopt a consistent output encoding strategy across the entire application, preferably through a templating engine that auto-escapes by default.
- Conduct periodic security scans to detect new XSS vectors.

6 - Broken Database Integrity

HIGH RISK (7/10)	
Exploitation Likelihood	Possible
Business Impact	Moderate
Remediation Difficulty	Easy

Security Implications

Unhandled foreign key constraint violations exposed database schema information and caused application errors, leading to logic failures and information disclosure.

Analysis

The vulnerable version did not properly validate data relationships before insert operations, leading to errors like: `Integrity constraint violation: 1452 Cannot add or update a child row.`

Status in Secure Version:

- Fixed by implementing pre-insert validation and proper exception handling.
- Database-level constraints are now respected and handled gracefully by the application.

Recommendations:

- Ensure all database operations are wrapped in proper try-catch blocks.
- Validate the existence of related entities (e.g., that a doctor exists) before creating dependent records (e.g., an appointment).

7 - Missing Input Validation

MEDIUM RISK (5/10)	
Exploitation Likelihood	Possible
Business Impact	Moderate
Remediation Difficulty	Easy

Security Implications

The backend accepted empty or missing POST fields, leading to undefined array key warnings, data inconsistency, and potential logic bypasses.

Analysis

Forms could be submitted with missing required fields, causing the application to throw warnings and behave unexpectedly.

Status in Secure Version:

- Fixed by enforcing required field validation on the server-side.
- All incoming JSON/POST data is now strictly validated and sanitized.

Recommendations:

- Implement a centralized input validation library to ensure consistency.
- Define and enforce strict data schemas for all API endpoints.

APPENDIX A - TOOLS USED

TOOL	DESCRIPTION
BurpSuite Community Edition	Used for intercepting, manipulating, and replaying HTTP requests to test for injection and logic flaws.
Postman	Used for crafting specific API requests and testing backend endpoints.
Web Browser Developer Tools	Used for analyzing client-side code, manipulating HTML forms, and debugging.

APPENDIX B - ENGAGEMENT INFORMATION

Client Information

Client	Hospital Management System Stakeholders
Primary Contact	Omar Magdy Team Leader
Approvers	The following people are authorized to change the scope of engagement and modify the terms of the engagement • Keroslos Zakarya • Abdelrahman Ashraf

Version Information

Version	Date	Description
1.0	29/11/2025	Initial report to client

Contact Information

Name	DEPI Grduation project Consulting
Phone	0000-000-0000
Email	@gmail.com

1 Burp Project Intruder Repeater View Help Burp Suite Community Edition v2025.10.4 - Temporary Project

Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Learn

Intercept HTTP history WebSockets history Match and replace Proxy settings

Intercept on Forward Drop

Response from http://localhost:80/DEPI_Final/Vulnerable/Info/patient/profile.php [127.0.0.1] Open browser

Time	Type	Direction	Method	URL	Status code	Length
1	POST	localhost	POST	/DEPI_Final/Vulnerable/Info/patient/profile.php	HTTP/1.1	

Request

```
1 POST /DEPI_Final/Vulnerable/Info/patient/profile.php HTTP/1.1
2 Host: localhost
3 Content-Length: 453
4 Cache-Control: max-age=0
5 sec-ch-ua: "Not A Brand",v="99", "Chromium",v="142"
6 sec-ch-ua-mobile: ?0
7 sec-ch-ua-platform: "Windows"
8 Accept-Language: en-US,en;q=0.9
9 Origin: http://localhost
10 Content-Type: application/x-www-form-urlencoded
11 Upgrade-Insecure-Requests: 1
12 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/142.0.0.0 Safari/537.36
13 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
14 Sec-Fetch-Site: same-origin
15 Sec-Fetch-Mode: navigate
16 Sec-Fetch-User: ?1
17 Sec-Fetch-Dest: document
18 Referer: http://localhost/DEPI_Final/Vulnerable/Info/patient/profile.php
19 Accept-Encoding: gzip, deflate, br
20 Cookie: PHPSESSID=mdcia50hph3yibh3e1efisxrh
21 Connection: keep-alive
22
23 User-Agent:
24 34b3e33723e556c6e49729309c2e6556d081a6596c93d84b655726e586c6e49739309c254343309
25 6786081e4596c6c2e46308f6e08c26e4461746556e5e6e6586e2c26597237468093d83233093
26 09245c20303c23c2032379268679656e464655723d83c4954d84792095335c24333d86a61
27 976e1372932372e597097493a616c6e587297492097795093639c792993e40d80a2e616c
28 6c6e16c6e4852748709706582083c4954d84792095335c24333d86a6170961373d8397236
29 597097493a616c6e587297492097795093639c792993e40d80a2e616c
```

Response

```
Warning: Undefined array key "email" in
D:\Xampp\htdocs\DEPI_Final\Vulnerable\Info\patient\profile.php
on line 29

Deprecated: trim(): Passing null to parameter #1 ($string) of type string
is deprecated in
D:\Xampp\htdocs\DEPI_Final\Vulnerable\Info\patient\profile.php
on line 29

Warning: Undefined array key "date_of_birth" in
D:\Xampp\htdocs\DEPI_Final\Vulnerable\Info\patient\profile.php
on line 30

Warning: Undefined array key "gender" in
D:\Xampp\htdocs\DEPI_Final\Vulnerable\Info\patient\profile.php
on line 31

Warning: Undefined array key "blood_type" in
D:\Xampp\htdocs\DEPI_Final\Vulnerable\Info\patient\profile.php
```

Inspector

- Request attributes: 2
- Request body parameters: 1
- Request cookies: 1
- Request headers: 20
- Response headers: 10

Event log All issues

20°C عالم

Search

Memory: 176.5MB Disabled

7:36 PM 28/11/2025

localhost/DEPI_Final/Vulnerable

localhost/DEPI_Final/Vulnerable/Info/patient/appointments.php

Fatal error: Uncaught PDOException: SQLSTATE[23000]: Integrity constraint violation: 1452 Cannot add or update a child row: a foreign key constraint fails ('medical_system`.`appointments', CONSTRAINT 'appointments_ibfk_2' FOREIGN KEY ('doctor_id') REFERENCES 'doctors' ('id') ON DELETE CASCADE) in D:\Xampp\htdocs\DEPI_Final\Vulnerable\Info\patient\appointments.php:38 Stack trace: #0 D:\Xampp\htdocs\DEPI_Final\Vulnerable\Info\patient\appointments.php(38): PDOStatement->execute(Array) #1 [main] thrown in D:\Xampp\htdocs\DEPI_Final\Vulnerable\Info\patient\appointments.php on line 38

20°C عالم

Search

ENG INTL

7:43 PM 28/11/2025

